

.NET / ASP.NET Core

Interview Preparation — Questions & Model Answers

26 scenario-based questions covering async/await, dependency injection, middleware pipeline security, API design, and production-grade coding patterns.

Each answer is written as a spoken, interview-ready paragraph.

Q1. What are the best practices for reducing the size of a large JSON API response, and for minimizing the number of API calls required to retrieve a significant amount of data?

Theory Concepts Tested

- DTO projection vs. serializing EF entities directly
- Offset pagination vs. keyset (cursor) pagination
- Response compression (Brotli/Gzip)
- JsonSerializerOptions.DefaultIgnoreCondition for nulls
- Sparse fieldsets / field selection
- Batching, bulk endpoints, BFF/GraphQL aggregation
- Caching (ETag, Cache-Control, distributed cache)
- Streaming large results with IEnumerable<T>

Model Answer (Interview-Ready)

To reduce payload size, the first step is to stop serializing EF entities directly and instead project into purpose-built DTOs with a `Select()` so the database only returns the fields actually needed, avoiding over-fetching navigation properties. Pagination is essential for any endpoint that can return an unbounded set of rows; for large or frequently changing datasets, keyset (cursor) pagination scales better than offset pagination because it avoids scanning past skipped rows. Enabling response compression middleware (Brotli or Gzip) is usually the single biggest win for JSON payloads since text compresses very well, and configuring `JsonSerializerOptions` to omit null fields avoids sending redundant "field": null entries. For clients that only need a subset of fields, supporting sparse fieldsets (e.g. `?fields=id,name`) lets different consumers request only what they need. On the side of minimizing API calls, the key is avoiding chatty client patterns — instead of one call per item, expose batch endpoints that accept a list of IDs, and instead of a client calling multiple endpoints to assemble one screen's worth of data, provide a composed endpoint or a BFF/GraphQL layer that aggregates on the server side. Caching, whether via HTTP caching headers like ETag or a distributed cache such as Redis, prevents unnecessary repeat calls for unchanged data. Finally, for very large exports, streaming the response using `IAsyncEnumerable<T>` avoids materializing the entire result set in memory and improves time-to-first-byte.

Q2. A controller action needs to process an uploaded image and persist the result to a database. How would you implement each part so that the request thread is never blocked?

Theory Concepts Tested

- `IFormFile.OpenReadStream()` and streaming uploads
- Async I/O vs. thread blocking
- `CopyToAsync` / `SaveChangesAsync`
- `Task.Run` misconceptions for I/O-bound work
- Long-running work: 202 Accepted + background processing

Model Answer (Interview-Ready)

The core principle is that 'not blocking the thread' means using genuinely asynchronous I/O rather than just moving work to another thread. For the upload itself, I would read the file via `IFormFile.OpenReadStream()` and stream it directly to storage with `CopyToAsync` rather than buffering the whole file into a `MemoryStream`, since `CopyToAsync` performs non-blocking I/O and releases the thread back to the pool while waiting on disk or network. For the database write, I would use `SaveChangesAsync()` rather than the synchronous `SaveChanges()`, since EF Core's async APIs use async

ADO.NET calls under the hood and don't block the thread waiting on the round trip. It's important not to reach for `Task.Run` here just to 'avoid blocking' — `Task.Run` is meant for CPU-bound work, and wrapping I/O-bound work in it doesn't reduce thread usage, it just moves the block to a different thread pool thread. If the image processing itself is heavy — resizing, virus scanning, thumbnail generation — I would avoid doing it inline in the request entirely: save the upload and a pending database record quickly, return 202 Accepted immediately, and enqueue the actual processing to a background job via a queue or an `IHostedService/BackgroundService`, updating the record's status asynchronously once it completes. That keeps the request thread free almost immediately instead of holding it for the full duration of processing plus the database write.

Q3. The following code works, but could be improved. What would you change and why?

```
public async Task<string> GetStatusAsync()
{
    var response = await _client.GetAsync("/status");
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadAsStringAsync();
}
```

Theory Concepts Tested

- `IHttpClientFactory` vs. manual `HttpClient` instantiation
- `CancellationToken` propagation
- `IDisposable` resource cleanup (using `using` on `HttpResponseMessage`)
- `EnsureSuccessStatusCode()` losing response body detail
- Resiliency (Polly retry/circuit-breaker policies)

Model Answer (Interview-Ready)

While this code functions correctly, there are several production-grade improvements worth making. First, I would confirm `_client` is obtained through `IHttpClientFactory` rather than instantiated with `new HttpClient()` per call, since the latter risks socket exhaustion under load. Second, I would add a `CancellationToken` parameter and pass it through to both `GetAsync` and `ReadAsStringAsync`, so a caller — or the ASP.NET Core request pipeline itself via `HttpContext.RequestAborted` — can cancel a hung or abandoned request instead of it running to completion regardless. Third, `HttpResponseMessage` is `IDisposable`, so I would wrap it in a `using` statement to ensure the underlying connection and stream resources are released promptly rather than relying on GC finalization. Fourth, `EnsureSuccessStatusCode()` throws away the response body on failure, which often contains the actual error detail from the downstream API — I would instead check `response.IsSuccessStatusCode` explicitly and include the response body in a custom exception message so failures are diagnosable. Finally, for genuine resiliency against transient network failures, I would layer in a Polly retry or circuit-breaker policy through `IHttpClientFactory`'s `AddPolicyHandler` rather than handling failure ad hoc inside this method.

Q4. The following code has a compile-time error. What is the error, how would you fix it, and why?

```
public class Customer
{
    public required string Name { get; set; }
```

```
public int Level { get; set; }  
}  
  
var c = new Customer(Level = 3);
```

Theory Concepts Tested

- Object-initializer syntax vs. constructor call syntax
- The `required` modifier (C# 11+) and compiler enforcement
- SetsRequiredMembers attribute
- required vs. nullable-reference-type warnings

Model Answer (Interview-Ready)

There are two layered problems here. First, `new Customer(Level = 3)` uses parentheses, which the compiler interprets as a constructor call taking one argument — `Customer` has no such constructor, so this doesn't compile as written; object initialization requires curly braces: `new Customer { Level = 3 }`. Second, even after fixing the braces, this still fails to compile because `Name` is marked required, and the required modifier — introduced in C# 11 — forces the compiler to enforce that the member must be explicitly assigned at the object-initializer call site, precisely so a mandatory property can never be silently left unset. The correct fix is `new Customer { Name = "Alice", Level = 3 }`. Alternatively, if the design should also support construction via a constructor, I would add a constructor decorated with `[SetsRequiredMembers]`, which tells the compiler that constructor guarantees all required members are set, allowing callers to use `new Customer("Alice", 3)` without needing object-initializer syntax at all. The key concept being tested is that required is a genuine compile-time enforcement mechanism, unlike nullable-reference-type annotations, which only produce warnings by default.

Q5. Review the following request pipeline. What security issue does this create?

```
app.UseStaticFiles();  
app.UseAuthentication();  
app.UseAuthorization();
```

Theory Concepts Tested

- Middleware pipeline execution order
- Static files bypassing authentication/authorization
- Correct canonical middleware ordering
- Protecting non-public content via authenticated endpoints instead of `UseStaticFiles`

Model Answer (Interview-Ready)

Middleware executes in exactly the order it is registered, and here `UseStaticFiles()` is registered before `UseAuthentication()` and `UseAuthorization()`. This means any request for a file served from `wwwroot` (or another configured static path) is handled and short-circuited by the static files middleware before the pipeline ever reaches the authentication or authorization checks. In practice, this means any private or sensitive content placed under a static file path — internal documents, user-uploaded files, admin UI bundles — is accessible to anyone with the URL, authenticated or not, because those requests never pass through the auth middleware at all. The fix is to reorder the pipeline so authentication and authorization run first, but it's important to recognize that reordering alone doesn't automatically protect static files, since `UseStaticFiles()` has no concept of `[Authorize]` attributes. The complete fix is therefore twofold: keep genuinely public assets like CSS, JS, and images served early through `UseStaticFiles()`, and serve

anything that needs to be protected through an authenticated controller or minimal API endpoint that reads the file from disk or storage and returns it via a `FileResult` decorated with `[Authorize]`. The broader takeaway is that middleware order defines the actual security boundary of the pipeline — anything placed before the authentication/authorization middleware effectively bypasses it.

Q6. The following controller action has a model binding error. What is the error, and how would you fix it?

```
[HttpPost]
public IActionResult Create([FromBody] Customer customer, [FromBody] Address address)
{
    return Ok();
}
```

Theory Concepts Tested

- Single-read nature of the HTTP request body stream
- Only one `[FromBody]` parameter allowed per action
- Wrapper DTO pattern for combining multiple objects in a request body
- `[FromBody]` vs `[FromQuery]` vs `[FromRoute]`

Model Answer (Interview-Ready)

ASP.NET Core only allows one parameter per action to be bound from the request body, because the body is a single stream that is deserialized once as a whole — there is no way for the framework to know how to split one JSON payload into two independently-bound complex objects. This produces a runtime `InvalidOperationException` when the action is invoked, stating that the action has more than one parameter specified from the request body; it is not caught at compile time. The correct fix is to introduce a single wrapper DTO that contains both objects, for example a `CreateCustomerRequest` class with `Customer` and `Address` properties, and bind that single object with `[FromBody]`, with the client sending one JSON object containing both nested structures. If the two pieces of data don't logically belong together in the same payload, an alternative is to keep only one parameter bound from the body and source the other from the route or query string instead, but when both are genuinely part of the same request payload, the wrapper DTO is the standard, correct pattern.

Q7. The following code involves Kestrel, IIS, and a reverse proxy. Explain what it does, and identify what is missing or risky about this configuration.

```
builder.Services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardedHeaders = ForwardedHeaders.XForwardedFor |
ForwardedHeaders.XForwardedProto;
});

var app = builder.Build();
app.UseForwardedHeaders();
```

Theory Concepts Tested

- Why Kestrel behind a reverse proxy sees the proxy's IP/scheme, not the client's
- X-Forwarded-For / X-Forwarded-Proto headers

- Middleware ordering — UseForwardedHeaders must run first
- KnownProxies / KnownNetworks and header-spoofing risk
- ForwardLimit for multi-hop proxy chains

Model Answer (Interview-Ready)

When Kestrel runs behind a reverse proxy such as IIS, Nginx, or a cloud load balancer, it only sees the connection as coming from the proxy, so by default `HttpContext.Connection.RemoteIpAddress` reflects the proxy's IP rather than the real client, and `Request.Scheme` shows `http` even if the client connected over `https`, because TLS was terminated upstream. This configuration tells ASP.NET Core to read the `X-Forwarded-For` and `X-Forwarded-Proto` headers the proxy sets and rewrite the request's IP and scheme accordingly via `UseForwardedHeaders()`. Two things are missing or risky here. First, this middleware must run very early in the pipeline — before authentication, HTTPS redirection, and HSTS — otherwise those components make decisions based on the wrong scheme or IP; the snippet doesn't show it positioned relative to the rest of the pipeline, which is the first thing I'd verify. Second, and more importantly from a security standpoint, `KnownProxies` or `KnownNetworks` isn't configured, and by default ASP.NET Core only trusts these headers from loopback addresses — if the actual reverse proxy isn't on loopback, the headers are silently ignored, and if someone works around this by clearing the trusted list entirely without restricting it, the application ends up trusting these headers from any upstream sender, which opens the door to IP or scheme spoofing that could bypass IP allowlists or HTTPS-only checks. The correct fix is to explicitly register the proxy's actual IP or subnet in `KnownProxies/KnownNetworks`, and if there are multiple proxy hops in the chain, to also increase `ForwardLimit` beyond its default of one hop.

Q8. How would you limit incoming client requests within a time window using ASP.NET Core's built-in rate limiting middleware?

Theory Concepts Tested

- `Microsoft.AspNetCore.RateLimiting` (built-in since .NET 7)
- Fixed window, sliding window, token bucket, and concurrency limiter algorithms
- Partitioned limiting (per-IP / per-user) via `RateLimitPartition`
- Middleware placement relative to authentication

Model Answer (Interview-Ready)

ASP.NET Core has had built-in rate limiting since .NET 7 via the `Microsoft.AspNetCore.RateLimiting` middleware, removing the need for third-party packages in common scenarios. I would register it with `AddRateLimiter` and define a policy — for example a fixed window limiter permitting a set number of requests per time window — then apply it either to specific endpoints via `RequireRateLimiting` or globally via a `GlobalLimiter`. There are several algorithms available depending on the traffic shape being protected against: fixed window is simplest but can allow bursts right at the window boundary; sliding window smooths that out by dividing the window into segments; token bucket allows controlled bursts by replenishing tokens at a steady rate; and a concurrency limiter caps how many requests can run simultaneously rather than how many occur per time period, which is useful for protecting resource-heavy endpoints like file uploads. If the limit needs to apply per client rather than globally, I would partition the limiter using `RateLimitPartition` keyed on something like the client's IP address or authenticated user ID, so each client gets its own independent bucket. By default, exceeding the limit returns 429 Too Many Requests. The placement of `UseRateLimiter()` in the pipeline matters: it generally belongs early so excess traffic is rejected before reaching expensive downstream

logic, though if different limits are needed for authenticated versus anonymous users, it needs to run after authentication so the user identity is available to partition on.

Q9. Explain middleware short-circuiting: how and why does an authentication/authorization middleware terminate the pipeline by setting a 401 Unauthorized status?

Theory Concepts Tested

- Middleware as chained RequestDelegates (chain of responsibility)
- Short-circuiting = not calling next()
- Authentication vs authorization middleware responsibilities
- 401 vs 403 semantics
- Why UseRouting must precede UseAuthorization

Model Answer (Interview-Ready)

ASP.NET Core middleware forms a pipeline in which each component receives a delegate representing the rest of the pipeline and decides whether to invoke it. Short-circuiting means a middleware chooses not to call that next delegate and instead writes a response directly, so nothing registered after it — including the controller action — ever executes. In the authentication/authorization flow specifically, the authentication middleware runs first and attempts to construct a ClaimsPrincipal from whatever credentials are present, such as a cookie or JWT, attaching it to HttpContext.User, but it generally still calls next() even if the resulting principal is unauthenticated. It is the authorization middleware, running after routing has selected an endpoint, that actually inspects that endpoint's [Authorize] metadata against HttpContext.User. If the user isn't authenticated at all, the authorization middleware sets the response status to 401 and returns without calling next(); if the user is authenticated but fails a role or policy check, it returns 403 instead. Either way, the request never reaches the controller, which serves both as a security boundary and a performance optimization, since none of the potentially expensive downstream logic executes for a request that was never going to be allowed. This is also exactly why middleware ordering is critical — anything registered before the authorization middleware still executes regardless of whether the request would ultimately be authorized.

Q10. Explain how injecting IEnumerable<T> allows resolving all registered implementations of an interface, rather than just the last one, when multiple implementations are registered with the DI container.

Theory Concepts Tested

- Default DI resolution behavior with multiple same-interface registrations
- IEnumerable<T> special-cased resolution in the built-in container
- Fan-out patterns: validators, notifiers, health checks
- Keyed services as an alternative when a specific implementation is needed

Model Answer (Interview-Ready)

By default, if multiple implementations of the same interface are registered with the DI container, resolving that interface directly only returns the last one registered — the container doesn't merge them or throw an error, it silently returns the most recent registration, which is an easy and subtle bug because the code compiles and runs without any error, it just quietly does the wrong thing. The fix is to inject IEnumerable<T> instead of T directly; the built-in .NET container has special handling for this case, so instead of resolving a single instance, it resolves all registered implementations of that interface, in registration order, as a collection. This is the standard pattern for fan-out

scenarios, such as running every registered `IValidator<T>` against an object, sending a notification through every registered channel, or — as a concrete real-world example — how ASP.NET Core's own health checks feature works internally, aggregating multiple `IHealthCheck` implementations. It's worth noting that if the actual goal is to select one specific implementation by name or context rather than run all of them, `IEnumerable<T>` isn't the right tool; that scenario calls for keyed services, introduced in .NET 8, or a factory pattern that resolves the correct implementation based on a key at runtime.

Q11. Explain how `ValidateScopes` and `ValidateOnBuild` help catch captive dependencies and missing service resolutions at startup, using `UseDefaultServiceProvider()`.

Theory Concepts Tested

- Captive dependency — a longer-lived service holding a shorter-lived one
- `ValidateScopes`: detecting singleton-depends-on-scoped at resolution time
- `ValidateOnBuild`: validating the entire DI graph at startup rather than lazily
- Development vs Production defaults for these flags

Model Answer (Interview-Ready)

By default, the DI container doesn't fully validate service registrations up front — certain classes of bugs only surface when a specific service is actually resolved at runtime, meaning they can hide until a particular code path executes in production. `ValidateScopes` detects captive dependencies: the scenario where a longer-lived service, typically a singleton, depends on a shorter-lived one, typically scoped, such as a `DbContext`. Without this check, the scoped instance gets captured inside the singleton and effectively lives forever, outside its intended scope, which can cause it to be shared incorrectly across requests and lead to threading issues or stale data. This check is already enabled by default in the Development environment but disabled in Production for performance reasons, which is an important detail — the safety net that normally protects you locally is switched off where the bug would actually cause damage. `ValidateOnBuild` goes a step further and validates the entire dependency graph immediately when the application builds, rather than only when each individual service happens to be resolved — so if any registered service has a dependency that cannot be resolved, or any lifetime violation exists anywhere in the graph, the application fails immediately at startup with a clear error message, instead of failing unpredictably later on a rarely-exercised code path. Enabling both flags during development and in CI turns an entire class of intermittent, hard-to-reproduce runtime bugs into deterministic, fail-fast startup errors, which is considerably cheaper to diagnose and fix.

Q12. When an API creates a new resource and returns HTTP 201 Created, how should it include the newly created resource and its location in the response?

Theory Concepts Tested

- 201 Created semantics vs 200 OK
- Location header per RFC 9110
- `CreatedAtAction` / `CreatedAtRoute` / `Created` helpers in ASP.NET Core
- Why route-based helpers are preferred over hardcoded URIs

Model Answer (Interview-Ready)

On successful creation of a resource, the API should respond with 201 Created rather than 200 OK, include a Location header pointing to the URI where the new resource can be retrieved, and ideally return the created resource itself in

the response body so the client isn't forced into an unnecessary follow-up GET just to see what was created. In ASP.NET Core, I would not construct this response manually — I would use `CreatedAtAction`, which takes the name of the action responsible for retrieving that resource, the route values needed to build its URL such as the newly generated ID, and the resource itself as the body; the framework then generates the Location header from actual routing metadata. `CreatedAtRoute` works the same way but references a named route instead of an action name, which is useful when action names might change independently of routes. There's also a `Created(uri, value)` overload for constructing the URI manually, but `CreatedAtAction` and `CreatedAtRoute` are generally preferred because they derive the URL from routing configuration rather than a hardcoded string, so if the route template changes later — for example during API versioning — the Location header updates automatically instead of silently pointing to a stale path. The common mistakes to avoid are returning 200 instead of 201 on creation, omitting the Location header entirely, and returning 201 with an empty body.

Q13. When serializing a C# object to JSON, why does a property named `CustomerName` (or `customerName`) become the JSON key "customerName"? What determines the JSON property name, and how can this behavior be controlled?

Theory Concepts Tested

- `System.Text.Json` default behavior (no casing transformation)
- ASP.NET Core's default `camelCase` `PropertyNamingPolicy`
- `[JsonPropertyName]` attribute overriding the global policy
- Case-insensitive deserialization in ASP.NET Core web defaults

Model Answer (Interview-Ready)

The C# property name itself doesn't dictate the JSON key — that transformation is controlled entirely by the `PropertyNamingPolicy` configured on `JsonSerializerOptions`. `System.Text.Json` by itself, used with no explicit configuration, preserves the property name exactly as written, so `CustomerName` would serialize as "CustomerName". The reason it commonly shows up as camelCase in an ASP.NET Core Web API is that `AddControllers()` configures camelCase as the default naming policy out of the box, purely as an ASP.NET Core convention layered on top of the serializer, not something `System.Text.Json` does inherently. If a specific property needs a different JSON key regardless of the global policy — for example matching a legacy contract using snake_case — the `[JsonPropertyName]` attribute can be applied directly to that property, and it takes priority over any global naming policy. If the global camelCase behavior needs to be disabled entirely to preserve exact C# casing, the `PropertyNamingPolicy` can be set to null in `AddJsonOptions`. It's also worth noting that deserialization in ASP.NET Core is case-insensitive by default, so incoming JSON keys bind correctly to properties regardless of casing — the naming policy really only governs outgoing serialization.

Q14. How should an API represent a model that contains an optional boolean flag property? Discuss how to model an optional boolean in the request model and its JSON representation.

Theory Concepts Tested

- `bool` always has a value (default false) — cannot represent 'unset'
- `bool?` distinguishing not-provided from explicitly-false
- PATCH/partial-update semantics

- System.Text.Json not distinguishing omitted vs explicit null by default
- When plain bool is still preferred (response models)

Model Answer (Interview-Ready)

A plain bool in C# can never be 'unset' — it always carries a value, defaulting to false — which creates ambiguity for API consumers, since a client omitting the field entirely and a client explicitly sending false both end up indistinguishable on the server. This matters most in PATCH-style partial update scenarios, where the intent is 'only update this field if the caller actually sent it, otherwise leave it unchanged' — with a plain bool there is no way to know which case occurred. The correct representation is a nullable bool, bool?, where null means 'not specified, leave as-is' and true/false represent an explicit value the client intended to set. It's worth flagging that System.Text.Json, by default, does not distinguish between a property being completely omitted from the payload versus explicitly sent as JSON null — both deserialize to null on a bool? — so if that finer distinction genuinely matters, which comes up in strict JSON Patch/Merge Patch semantics, something more explicit is needed, such as tracking which keys were actually present in the raw payload. For response models, though, I would lean back toward a plain bool unless there's a genuine third state like 'not yet calculated' that is meaningfully different from true or false, since introducing unnecessary nullability just forces API consumers to handle a null case that shouldn't really exist.

Q15. Consider the following C# method — what are the problems (compile-time, runtime, and logical) with this code?

```
public async Task DoProcess()
{
    var obj = await somemethod();
    await DoProcess();
}
```

Theory Concepts Tested

- Unbounded recursion with no base case
- StackOverflowException characteristics (uncatchable, terminates process)
- Unused variable as a signal of an incomplete exit condition
- Recursion vs. loop for repeated async operations
- Missing cancellation and error handling

Model Answer (Interview-Ready)

This method calls itself unconditionally on every invocation with no exit condition at all, which means it recurses indefinitely; even though each call is awaited, each await still schedules a continuation, so the recursion keeps consuming resources until the process eventually crashes with a StackOverflowException — an exception that cannot be caught in .NET and terminates the process immediately, making this the single most serious problem in the snippet. A secondary but very telling issue is that obj is assigned from somemethod() but never used anywhere in the method body; given the structure of the code, this is almost certainly meant to serve as the exit condition — for example, checking whether obj is null before recursing again — so this looks like an incomplete refactor rather than intentional design. Even once a proper exit condition is restored, I would argue this should be expressed as a loop rather than recursion in the first place, since a while or do-while loop communicates 'keep doing this until a condition is met' far more clearly and avoids building up an ever-growing chain of continuations. Beyond that, the method has no CancellationToken, so if this represents some kind of polling or repeated processing operation, there is no way for a

caller to stop it gracefully, and there is no error handling, so any exception thrown by `somemethod()` propagates up and terminates the entire operation. Strictly speaking, this code does compile — the unused variable only produces a compiler warning, not an error — so the real problems here are runtime and design-level rather than genuine compile-time failures.

Q16. Consider the following repository service code — what are the problems (syntax, typing, null-safety, naming, and design) with this implementation?

```
public class RepositoryService
{
    private readonly _repository;
    public async Task<Order>(int id)
    {
        var obj = await _repository.Find(id);
        return new Order() { Name = obj!.Name, Id = obj!.Id };
    }
}
```

Theory Concepts Tested

- Field declarations require an explicit type
- Method declarations require a name (`Task<Order>` alone is not valid syntax)
- `readonly` fields require constructor assignment for DI
- Null-forgiving operator (!) suppresses warnings but not runtime null risk
- Naming conventions (`_repository`, `Find`, `obj` all too vague)
- Mapping design: constructing a new object vs. returning/mapping the source properly

Model Answer (Interview-Ready)

This snippet has genuine compile errors in addition to design smells. The field `private readonly _repository;` is missing a type entirely, which will not compile, and the method signature `public async Task<Order>(int id)` is missing a method name — `Task<Order>` is being used as though it were the method's name, which is also invalid syntax; it needs to read as `public async Task<Order> GetByIdAsync(int id)`. There is also no constructor, so even once a type is added, the `readonly _repository` field could never be assigned, and dependency injection would have no way to populate it either. Setting the syntax aside, the null-safety here is genuinely concerning: the code uses the null-forgiving operator, `obj!`, to suppress the compiler's nullable-reference warning, but that operator has no effect at runtime — if the repository genuinely returns null because the record wasn't found, this throws a `NullReferenceException` instead of handling the missing-record case explicitly; I would replace it with an explicit null check that throws a meaningful exception like `KeyNotFoundException` or returns null, depending on the calling convention. On naming, nearly everything here is too vague to be maintainable — `_repository`, `Find`, `obj`, and even the class name `RepositoryService` itself give no indication of what domain object is being worked with, and should be renamed to something like `_orderRepository`, `FindAsync`, and `orderEntity` respectively. From a design perspective, I would also question why the method manually reconstructs a new `Order` object copying only two fields from what is presumably already an `Order`-shaped source — if this is genuinely mapping between an entity and a DTO, that mapping should go through a proper, explicit mapping method or library rather than a hand-rolled initializer that could silently drop fields the target type actually needs.

Q17. The following code contains a common async anti-pattern. What is the issue, and how would you fix it?

```
public IActionResult GetData()
{
    // Issue: Blocking the thread. Leads to deadlocks in UI or ASP.NET apps.
    var data = _service.GetAsync().Result;
    return Ok(data);
}
```

Theory Concepts Tested

- Sync-over-async blocking via .Result
- SynchronizationContext deadlock mechanism (classic ASP.NET / UI apps)
- Why ASP.NET Core is less deadlock-prone but still at risk of thread pool starvation
- AggregateException wrapping vs. direct exception propagation with await
- The 'async all the way' principle

Model Answer (Interview-Ready)

Calling .Result on an async method synchronously blocks the calling thread until that task completes, which defeats the purpose of using async in the first place and introduces two distinct risks. The first is deadlock: in an environment with a synchronization context, such as classic ASP.NET Framework or a WPF/WinForms UI thread, the awaited method tries to resume back on that same original context once its internal await completes, but that context is occupied because it's blocked waiting on .Result, so neither side can make progress and the request hangs indefinitely. ASP.NET Core specifically does not have a synchronization context by default, so this exact deadlock is less likely to occur here, but the pattern is still dangerous because the same code, if reused inside a shared library later called from a context that does have one, would deadlock immediately. The second risk, which very much does apply in ASP.NET Core, is thread pool starvation: every request that blocks like this ties up a thread pool thread doing nothing but waiting instead of returning it to serve other requests, and under sufficient concurrent load this can exhaust the thread pool and cause the whole application to slow to a crawl or stop responding. There's also a secondary difference in exception handling — .Result wraps exceptions in an AggregateException, whereas await unwraps and rethrows the original exception directly, which can be confusing if calling code expects to catch a specific exception type. The fix is to go async all the way through: make the action async Task<IActionResult> and await _service.GetAsync() directly, rather than blocking on the result partway up the call chain.

Q18. How would you unit test a service that calls an external API without making a real HTTP request?

Theory Concepts Tested

- HttpResponseMessage as the true seam for testing HttpClient
- Hand-rolled fake HttpResponseMessage vs. RichardSzalay.MockHttp
- Preferable architecture: wrapping HttpClient behind a custom interface
- Integration tests (WireMock.Net / WebApplicationFactory) as a complementary, separate test tier

Model Answer (Interview-Ready)

The key is to isolate the HTTP dependency so the test never actually reaches the network. HttpClient itself isn't easily mockable directly, but the HttpResponseMessage it delegates to is the actual extension point, so I can write a small fake handler that returns a canned HttpResponseMessage regardless of the request, wire it into a real HttpClient, and inject that into the service under test — the service then runs exactly as it would in production, just without a real network

call underneath it. More commonly in practice, I would reach for a library like `RichardSzalay.MockHttp`, which lets you set up expected requests and matching canned responses fluently and then hands back a real `HttpClient` wired to that fake handler, which reads more cleanly than a hand-rolled handler class. That said, my preferred architecture is to wrap the external call behind a small interface, such as `IStatusApiClient`, so that any business logic depending on it can be tested with a completely standard mock from `Moq` or `NSubstitute` without touching `HttpMessageHandler` mechanics at all — the thin wrapper around `HttpClient` becomes the only piece that needs handler-level testing, and it's usually simple enough to cover with just a couple of focused tests, or with an integration test instead. For genuine end-to-end confidence beyond unit tests, I would use something like `WireMock.Net` or `WebApplicationFactory` to spin up a fake server and verify the actual wire-level contract, but that belongs to a separate integration testing tier, not the unit test layer.

Q19. What is the purpose of middleware in ASP.NET Core, and when would you choose to write custom middleware versus using an action filter or inline logic?

Theory Concepts Tested

- Middleware as a chain-of-responsibility pipeline over `HttpContext`
- Cross-cutting concerns centralized once vs. duplicated per-action
- Order-dependence of the pipeline
- When to prefer action filters or business-layer logic instead of middleware

Model Answer (Interview-Ready)

Middleware components are chained together into a pipeline through which every HTTP request passes; each component receives the `HttpContext`, performs some work, and either calls the next middleware in the chain or short-circuits by writing a response directly. The purpose of this design is to centralize cross-cutting concerns that every request needs regardless of which specific endpoint handles it — authentication, exception handling, logging, CORS, response compression, routing, and rate limiting are all handled once in the pipeline rather than duplicated as repeated logic inside every controller action. Order matters critically here, since middleware executes in exactly the sequence it's registered, which is why, for example, authorization must be registered after routing has selected an endpoint and before the endpoints it's meant to protect. I would reach for custom middleware specifically when I have logic that genuinely applies broadly across most or all requests and doesn't naturally belong to a single endpoint — examples include injecting a correlation ID for distributed tracing, logging every request's method, path, and response time, or validating a custom API key before any endpoint executes. If the logic is specific to only one or a handful of endpoints, I would use an action filter or put the logic directly in the action instead, since forcing endpoint-specific logic into middleware, which runs for every request by default, usually leads to messy conditional branching to scope it down. Business or domain logic belongs in services, not middleware, which should stay thin and focused purely on infrastructure-level, cross-cutting concerns.

Q20. What would you change in the following exception-handling code, and why?

```
try
{
    // some logic
}
```

```
catch (Exception ex)
{
    _logger.LogError(ex.Message);
    throw ex;
}
```

Theory Concepts Tested

- throw ex; resets the stack trace vs. throw; preserving it
- ILogger overload accepting Exception vs. logging only ex.Message
- Structured logging with message templates
- Whether a catch-log-throw block is even necessary alongside global exception handling

Model Answer (Interview-Ready)

There are two well-known mistakes here. First, throw ex; resets the stack trace to this line, so when the exception is eventually logged or surfaced further up the call stack, all information about where it actually originated is lost, which makes debugging production issues significantly harder; the fix is throw; with no expression, which rethrows the same exception object while preserving its original stack trace, simply adding this frame on top of it. Second, _logger.LogError(ex.Message) only logs the message string, discarding the exception's type, its full stack trace, and any inner exceptions, which are frequently where the actual root cause lives, especially with wrapped exceptions like a database driver exception nested inside a higher-level one; the fix is to use the ILogger overload that accepts the Exception object itself as the first argument, along with a structured message template, so the logging provider captures the complete exception details and any structured fields can be queried later rather than being buried in a flattened string. Beyond those two fixes, I would also ask whether this catch-and-throw block is even necessary in the first place — if the application already has centralized exception-handling middleware that logs unhandled exceptions, logging here as well may just be creating duplicate log entries for the same failure, unless this specific layer is adding real value, such as enriching the exception with additional context or wrapping it into a more specific domain exception type before rethrowing.

Q21. What is the difference between IOption<T> and IOptionSnapshot<T> in ASP.NET Core?

Theory Concepts Tested

- IOption<T> — singleton lifetime, read once and cached for the app's lifetime
- IOptionSnapshot<T> — scoped lifetime, re-read once per request
- Captive dependency risk: IOptionSnapshot<T> cannot be injected into a singleton
- IOptionMonitor<T> as the singleton-safe, live-reloading alternative

Model Answer (Interview-Ready)

Both are part of the options pattern for strongly typed configuration, but they differ in lifetime and in when the underlying value gets refreshed. IOption<T> is registered as a singleton and reads its configuration value once, the first time it is resolved, then caches that value for the entire lifetime of the application — so if the underlying configuration source changes at runtime, IOption<T> will not reflect that change without an application restart. IOptionSnapshot<T> is registered as scoped instead, and re-reads the configuration once per scope, which in a web application effectively means once per HTTP request; if the configuration source supports change notifications, such as appsettings.json with reload-on-change enabled, a subsequent request will automatically see the updated values without a restart. The important gotcha is that because IOptionSnapshot<T> is scoped, it cannot be injected into a singleton service — doing so is the same captive dependency problem as injecting any scoped service into a singleton,

and it will throw at resolution time, or be caught earlier if `ValidateScopes/ValidateOnBuild` are enabled. If live-reloading configuration is needed specifically inside a singleton or a background service, I would use `IOptionsMonitor<T>` instead, which is itself registered as a singleton and therefore safe to inject into other singletons, and which supports an `OnChange` callback that fires immediately whenever the underlying configuration changes, rather than only being refreshed per-request like `IOptionsSnapshot<T>`.

Q22. What issues are present in the following code, and how would you fix them?

```
public async Task<string> GetString(int id)
{
    using var httpClient = new HttpClient();
    await httpClient.GetStringAsync("https://api.example.com/items");
}
```

Theory Concepts Tested

- Missing return statement (compile error)
- `new HttpClient()` per call — socket exhaustion / `TIME_WAIT`
- `IHttpClientFactory` / typed clients as the correct pattern
- Unused method parameter (`id` never used in the request URL)
- Missing cancellation and error handling

Model Answer (Interview-Ready)

As written, this code will not even compile: the method is declared to return `Task<string>`, but it awaits `GetStringAsync` and discards the result without ever returning it, which is a straightforward 'not all code paths return a value' compile error — the fix is simply to add `return` before the `await`. The more significant issue, once that's fixed, is creating a new `HttpClient` inside the method and disposing it immediately afterward via `using`. `HttpClient` is designed to be long-lived and reused, not created and torn down per call; disposing it doesn't immediately release the underlying socket, which can linger in a `TIME_WAIT` state, and under any real load, repeatedly creating and disposing clients like this can exhaust available sockets and start causing connection failures. The correct fix is to obtain the client through `IHttpClientFactory`, typically via a typed client registered with `AddHttpClient`, which manages connection pooling correctly without either the socket-exhaustion problem of creating one per call or the DNS-staleness problem of a single static client held forever. I would also point out that the `id` parameter is never actually used — the URL is hardcoded, so every call returns the same data regardless of what `id` is passed, which is very likely a bug relative to the method's intent and should instead be interpolated into the request path. Finally, there's no cancellation token and no error handling around the call, both of which I'd add depending on how this method is used further up the call chain.

Q23. Explain the role of `IConfiguration` in .NET. How does it support multiple configuration providers?

Theory Concepts Tested

- `IConfiguration` as a unified key-value abstraction over disparate sources
- Provider chain: JSON files, environment-specific overrides, user secrets, environment variables, command-line args
- Last-registered-provider-wins precedence for duplicate keys
- Binding to strongly-typed options rather than using raw string keys

- reloadOnChange and its relationship to IOptionSnapshot/IOptionMonitor

Model Answer (Interview-Ready)

IConfiguration is the abstraction .NET uses to represent application settings as a single, unified key-value structure regardless of where the values actually originate — application code simply asks for a key, without needing to know whether it's backed by a JSON file, an environment variable, or a command-line argument. Underneath, configuration is built from a chain of providers added in a specific order, and each provider contributes key-value pairs into one merged structure; if the same key is defined by more than one provider, whichever provider was added last wins. In a typical ASP.NET Core application, that order is roughly appsettings.json for base defaults, then an environment-specific file like appsettings.Production.json, then user secrets in development, then environment variables, and finally command-line arguments, with each later source able to override the ones before it. This layering is the entire point of the design — it allows the same codebase to run correctly across local development, staging, and production, since the base file defines sensible defaults and each environment overrides only what it needs to, typically via environment variables injected by the deployment pipeline rather than by maintaining separate checked-in files per environment. Beyond the built-in file, environment variable, and command-line providers, additional sources like Azure Key Vault for secrets or Azure App Configuration for centralized settings across services can be added, and custom providers can be written for arbitrary sources such as a database table. While IConfiguration itself deals in flat string keys separated by colons for hierarchy, in application code I would almost always bind a configuration section to a strongly typed class through the options pattern rather than reading raw string keys throughout the codebase, and I'd note that providers supporting reloadOnChange are what allow IOptionSnapshot<T> and IOptionMonitor<T> to pick up configuration changes at runtime.

Q24. Review the following code. What is inefficient about it, and how would you improve it?

```
if (dictionary.ContainsKey(key))
{
    var value = dictionary[key];
    // do something with value
}
```

Theory Concepts Tested

- Double hashing/lookup: ContainsKey + indexer
- TryGetValue as a single-lookup replacement
- Thread-safety nuance: race condition between ContainsKey and indexer access
- GetValueOrDefault for simple fallback scenarios

Model Answer (Interview-Ready)

This code performs two separate lookups into the dictionary for the same key — ContainsKey hashes the key and walks the bucket to confirm existence, and the indexer then hashes the same key and walks the bucket again just to retrieve the value. Both operations are O(1) on average, so this isn't algorithmically catastrophic, but it's doing the hashing and bucket traversal twice for something that can be accomplished in a single lookup. The fix is TryGetValue, which checks for existence and retrieves the value in one operation, returning a boolean indicating whether the key was found and populating the value through an out parameter if it was — this is both faster and the standard, idiomatic C# pattern. There's also a subtler reason to prefer it: in a multi-threaded context where the dictionary is shared and mutated across threads, ContainsKey followed by the indexer are two distinct operations, so another thread could remove the key in

the window between them, causing a `KeyNotFoundException` even though the key was just confirmed to exist a moment earlier; `TryGetValue` avoids this specific race because it's effectively a single call from the caller's perspective, though it doesn't make the dictionary itself fully thread-safe — a `ConcurrentDictionary` would still be needed for genuine concurrent access. If the goal is simply 'give me the value or a fallback if it's missing,' `GetValueOrDefault` is even more concise for that specific read-only case.

Q25. In an ASP.NET Core Web API, how would you implement a centralized, uniform exception handling strategy?

Theory Concepts Tested

- `IExceptionHandler` (built-in since .NET 8) vs. `UseExceptionHandler` lambda vs. hand-rolled middleware
- `ProblemDetails` (RFC 9457) as a standardized error response shape
- Mapping exception types to appropriate HTTP status codes
- Never leaking exception details/stack traces to clients in production
- This complements, not replaces, model validation (`[ApiController]` / `FluentValidation`)

Model Answer (Interview-Ready)

The goal is to avoid try/catch scattered across every controller action, with inconsistent error shapes and duplicated logging, and instead handle all unhandled exceptions in one place. Since .NET 8, the recommended approach is implementing `IExceptionHandler`, registering it with `AddExceptionHandler` and `AddProblemDetails`, and then calling `UseExceptionHandler()` in the pipeline, which invokes the handler for any exception not caught elsewhere; multiple handlers can be registered and are tried in order, giving a clean way to have handlers for specific exception types plus a catch-all fallback. Inside the handler, I would map exception types to appropriate HTTP status codes — a validation exception becomes 400, a not-found becomes 404, an authorization failure becomes 403, and everything else falls back to 500 — and return the response in the standard `ProblemDetails` shape, a well-known format defined by RFC 9457 that many clients and tools already understand, rather than inventing a custom error structure per project. Before .NET 8, the equivalent was `UseExceptionHandler` with a lambda or fallback endpoint, or hand-rolled middleware wrapping `next()` in a try/catch — functionally similar, just less standardized. Two things I'd be careful about regardless of which approach is used: never leak the raw exception message or stack trace to the client in production, only in development, while always logging the full exception details server-side regardless of what's shown to the client; and remember that this mechanism is for unhandled runtime exceptions specifically, not a replacement for proper request validation, which `[ApiController]`'s automatic model-state handling and something like `FluentValidation` should already be catching before the action even executes.

Q26. In an e-commerce application, a developer stores the shopping cart as an instance field inside a class registered as a Singleton. What are the problems with this?

Theory Concepts Tested

- Singleton lifetime = one shared instance for the entire application process
- Shared mutable state across all users when data is stored on a singleton
- Separate issue: `List<T>` is not thread-safe under concurrent access
- Correct pattern: keep per-user data in Session, distributed cache, or a database, keyed by user/session ID

Model Answer (Interview-Ready)

A singleton service is created once for the lifetime of the application, and that exact same instance is shared by every request, every user, and every thread that hits the app. If the shopping cart is stored as an instance field directly on that singleton, there is effectively only one cart in the entire application — every user adding items is adding to the same underlying list, and every user retrieving their cart sees everyone's combined items, which is a serious, immediately visible functional bug rather than a subtle edge case. Separately from the shared-state problem, a plain `List<T>` is not thread-safe, so under real concurrent traffic — which is exactly what a web application experiences — multiple simultaneous calls to add items from different requests can corrupt the list's internal state, throw exceptions, or silently drop entries; this is a distinct bug from the lifetime issue, and fixing one does not automatically fix the other. The root cause is almost always registering the service as a singleton for a reasonable-sounding but incorrect reason, such as wanting 'only one instance for performance,' without recognizing that a singleton means one instance for the entire process, not one instance per user. The correct fix is to recognize that cart data is inherently per-user state and needs to live somewhere naturally scoped to the user rather than to the application — options include ASP.NET Core's session for simpler apps, a distributed cache like Redis when the application runs across multiple server instances behind a load balancer, or a database when the cart needs to persist across logins and devices. The service class itself can remain a singleton so long as it holds no per-user state and instead operates on data fetched externally by user ID — the actual mistake is putting mutable, user-specific data directly on a singleton, not the singleton lifetime of the service class itself.
